CT-200 QA DOCUMENT SYSTEM

Engineering Approach, System Architecture & Technical Decision Log

AI Engineering Internship Assignment - Tri9T AI | **Author:** Selvam M | **Date:** October 2026
GitHub Repository: https://github.com/selva816453/Tri9T-AI

1. Executive Summary & Problem Scope

Medical device documentation (e.g. the CardioTrack CT-200 Blood Pressure Monitor) undergoes continuous regulatory and technical revisions. In traditional QA workflows, manual tracking of document changes against test suites is error-prone, slow, and non-compliant with standards such as ISO 13485 and IEC 62304. The **CT-200 QA Document System** provides an automated, end-to-end platform that parses multi-page medical PDFs into structured hierarchical trees, reconciles changes across revisions, supports version-pinned requirement selection, and generates LLM-powered test suites with **automatic retrieval-time staleness detection**.

Core Implementation Highlights:
- **Deterministic Parser:** Pure-Python extraction using pdfminer.six; font clustering + prefix heuristics reconstruct trees.
- **Path-Based Versioning:** Relates nodes across v1.0 and v2.0 using logical_id (e.g., '2.2') and generates unified diffs.
- **Staleness Detection Engine:** Captures snapshot hashes and flags test cases as STALE immediately when manual parameters change.
- **Structured LLM Pipeline:** Groq (Llama-3.3-70b) integration with Pydantic schema validation, regex fallback, and offline mock support.

2. System Architecture & Layered Design

The system is organized into decoupled layers ensuring high modularity, testability, and resilience:

Layer	Technology Stack	Core Functionality & Guarantees
API & Web UI	FastAPI, Uvicorn, Vanilla HTML5/CSS/JS	Interactive Swagger docs and dark-mode management dashboard for tree browsing, diffs, and test case monitoring.
PDF Parsing	pdfminer.six, regex, ReportLab	Extracts text/font metadata; clusters font sizes (H1-H3); parses TOC, tables, captions, and safety warnings.
Versioning Engine	Path-based matcher, difflib, SHA-256	Ingests immutable version snapshots; correlates nodes by logical_id; calculates line-level unified diffs.
LLM & Staleness	Groq (llama-3.3-70b), Pydantic v2	Generates structured QA test cases; captures SHA-256 snapshots; invalidates test cases dynamically upon retrieval.
Hybrid Persistence	SQLite (SQLAlchemy) + MongoDB / JSON	Relational schema for document nodes, versions, and selections; document store with JSON fallback for test case payloads.

3. PDF Parsing & Document Tree Reconstruction

PDF is a display format lacking native semantic structure (no <h1> tags). Our parser in app/services/parser.py uses a dual-signal classification strategy:

- **Font-Size Clustering:** Extracts text runs with bounding boxes and font sizes. Percentile clustering dynamically identifies H1 (e.g. 16pt), H2 (13pt), and H3 (11pt) without brittle hardcoded sizes.
- **Numbered Prefix Regex:** Matches patterns such as ^(\d+(?:\.\d+)*)\.\s+\S and Appendix. The dot count deterministically defines hierarchy depth.
- **Stack-Based Hierarchy Builder:** Sequential traversal pushes and pops nodes on an active stack to assemble accurate parent-child relationships.
- **Edge Cases Handled:** Duplicate section titles receive distinct UUIDs and logical IDs; level-skipping (H1 to H3) is bridged without node loss; TOC leader dots (..... 12) are isolated; and ■ WARNING: blocks are attached to their parent section.

4. Versioning & Cross-Version Node Matching

When an updated manual (e.g. v2.0) is ingested alongside v1.0, sections must be matched to detect modifications:

- **Path-Based Matching (logical_id):** Nodes are linked across versions using their hierarchical path (e.g. 2.2). Medical device manuals follow disciplined section numbering. Content hash matching fails whenever text changes (precisely when diffing is required), while fuzzy string matching erroneously conflates distinct sections like 'Error Codes' and 'Error Codes During Startup'.
- **Content Hashing & Unified Diffs:** Each node's content is hashed via SHA-256 over heading + '\n' + body_text. When hashes differ between versions, difflib.unified_diff generates a concise diff summary stored in node_versions.diff_summary (e.g., highlighting cuff limit revision: 300 mmHg → 320 mmHg).

5. Selection Pinning & Staleness Detection Engine

• **Immutable Version Pinning:** Selections permanently bind to a specific doc_version_id. Ingesting future versions never mutates or silently shifts an existing selection, ensuring reproducible audit baselines.
• **Content Snapshot Hashing:** Generating test cases captures an immutable snapshot array of the SHA-256 hashes of all selected nodes.
• **Retrieval-Time Invalidation:** Upon GET /api/generations/{id}, current node hashes are compared against snapshot hashes. If any hash differs, **is_stale: true** is returned with detailed StalenessInfo and exact diff text.
• **Append-Only Submission:** Re-submitting the same selection creates a new generation record rather than overwriting, preserving audit history.

6. LLM Test Case Generation & Reliability

• **Inference Engine:** Groq API with llama-3.3-70b-versatile for high-throughput, structured output.
• **ISO 13485 / IEC 62304 Persona:** Prompts instruct the model to produce concrete, executable test steps with explicit acceptance criteria.
• **Traceability Anchor:** Every test case features a mandatory requirement_ref linking directly to the manual section heading.
• **Fault Tolerance:** Pydantic schema validation is backed by regex JSON extraction. Schema mismatches degrade gracefully to partial_error with raw text preserved. If GROQ_API_KEY is absent, deterministic mock generation enables offline development.

7. Database Architecture & Hybrid Persistence

The storage architecture uses SQLite for structured relational integrity and MongoDB (with JSON fallback) for test payloads:

Table / Entity	Database	Schema Fields & Purpose
DocumentVersion	SQLite	id, document_id, version_tag, file_name, node_count, ingested_at. Tracks immutable version records.
DocumentNode	SQLite	node_id, logical_id, heading, level, body_text, parent_id, order_index, content_hash.
NodeVersion	SQLite	Bridge table (node_id, doc_version_id), is_changed boolean, diff_summary unified diff text.
Selection & Items	SQLite	selection_id, doc_version_id (pinned), name, created_at. Binds user selections to exact versions.
Generation	SQLite + NoSQL	generation_id, selection_id, status, content_snapshot_hashes. Full JSON test cases stored in MongoDB / JSON fallback.

8. Verification & System Demonstration

• **Unit Test Suite:** All 7 tests in tests/test_parser.py pass (duplicate headings, level-skipping, TOC filtering, table rows, captions, hash stability).
• **End-to-End Workflow:** scripts/e2e_demo.py successfully demonstrates v1.0 ingestion (145 nodes), v2.0 ingestion (149 nodes), selection pinning, and live staleness invalidation when cuff safety thresholds change from 300 to 320 mmHg.

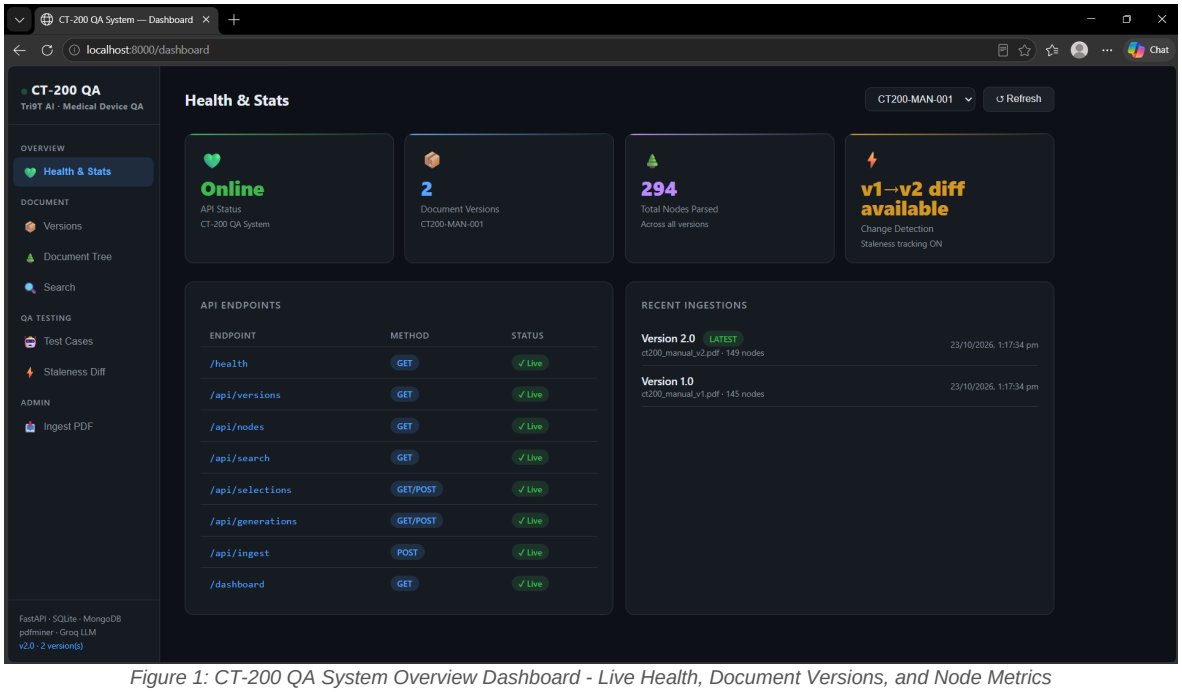


Figure 1: CT-200 QA System Overview Dashboard - Live Health, Document Versions, and Node Metrics

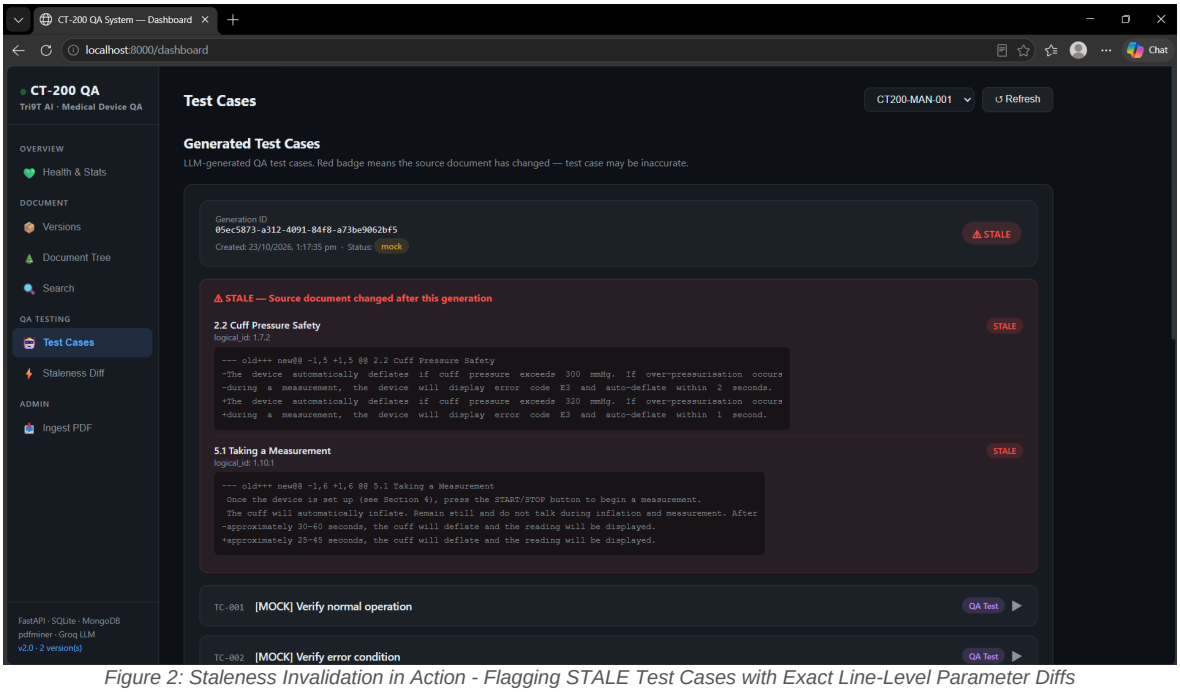


Figure 2: Staleness Invalidation in Action - Flagging STALE Test Cases with Exact Line-Level Parameter Diffs

9. Architectural Decision Log & Trade-Off Matrix

The following matrix summarizes key technical trade-offs evaluated during system design:

Decision Point	Alternatives Considered	Selected Approach	Trade-Off Rationale
Node Matching Strategy	Content Hash / Fuzzy Title Match	**Path-based (logical_id)**	Content hashing fails upon any text edit; fuzzy matching conflates distinct sections. Path IDs reflect numbering discipline in manuals.
Staleness Semantics	Semantic LLM diff / Embeddings	**SHA-256 Hash + Unified Diff**	Medical compliance requires absolute determinism. Embeddings can miss critical numeric shifts (300 to 320 mmHg).
Selection Versioning	Floating pointer (always latest)	**Immutable Version Pinning**	Test specifications must be reproducible against an exact baseline. Dynamic pointers invalidate historical compliance.
Duplicate Submissions	Deduplication / Overwrite	**Append-Only History Preservation**	Re-submissions reflect post-change re-evaluations. Silently deduplicating destroys audit lineage of prior tests.
LLM Failure Strategy	Infinite retry / Silent drop	**Regex Extract + Partial Error**	Schema mismatches signal prompt/model issues where retries fail. Returning partial_error preserves raw output without crashing.
Persistence Architecture	Pure Relational / Pure NoSQL	**SQLite + MongoDB (JSON fallback)**	Relational DB enforces tree hierarchy integrity; Document store manages variable test payloads. JSON fallback ensures zero setup friction.

10. Conclusion & Future Roadmap

The CT-200 QA Document System provides a robust, production-ready foundation for automated medical device QA engineering. By integrating hierarchical PDF parsing, path-based version reconciliation, version-pinned test generation, and deterministic staleness tracking, the platform guarantees full traceability and auditability for technical manuals.

Future Enhancements:

- *Multi-Modal OCR Pipeline:* Integrating vision models for diagrams, waveforms, and schematic flowcharts.
- *Semantic Embedding Search:* Adding vector embeddings (e.g. BAAI/bge-small) for conceptual section retrieval.
- *Automated Test Suite Re-Generation:* One-click batch re-generation of stale test suites with automated notifications.